



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

ESCUELA DE POSGRADO

Proyecto Parcial: FinZen
Aplicación de Finanzas Personales con Consultas Inteligentes (IA)

AUTOR(ES)

Sebastian Rodrigo García Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

DOCENTE

Mejia Fernandez, Oscar Rodolfo

CURSO

Cloud Computing (MCD8007)

Lima – Perú

2026

Índice

1. Introducción y resumen ejecutivo	2
Parte A: Análisis (QUÉ)	3
2. Empresa y producto propuesto	3
3. Benchmark — Comparación con la competencia	3
4. Alcance funcional de FinZen (100 % digital)	3
5. Propuesta de valor	4
6. Historias de Usuario y criterios de aceptación (Gherkin/BDD)	4
6.1. HU1 (MVP): Registro y Login	4
6.2. HU2 (MVP): Registrar Ingreso / Sueldo	5
6.3. HU3 (MVP): Registrar Gasto por categoría	5
6.4. HU4 (MVP): Listado y detalle de Ingresos	6
6.5. HU5 (MVP): Listado y detalle de Gastos	6
6.6. HU6 (MVP): Editar Perfil de Usuario	7
6.7. HU7 (MVP): Consulta Inteligente de finanzas (IA / embeddings)	7
6.8. HU8 (MVP): Dashboard con resumen financiero	8
Parte B: Diseño del MVP (CÓMO)	8
7. Diseño de FrontEnd (Prototipo) — Next.js	8
8. Diseño de BackEnd — FastAPI en contenedores Docker	9
8.1. Estructura de la base de datos (SQL — SQLite)	9
8.2. Selección del motor de base de datos: SQLite frente a PostgreSQL	13
9. Empaquetado con Docker	13

10.Diagrama de Arquitectura de Solución en AWS	14
10.1. Red en AWS (VPC, Internet Gateway, subred y Security Group)	15
11.Selección de región AWS	16
12.Costos mensuales estimados en AWS	16
12.1. Costos de OpenAI (aparte de AWS)	17
Parte C: Implementación del MVP	17
13.Catálogo de APIs	17
14.Implementación del BackEnd	18
15.Evidencia de uso con Postman	19
16.FrontEnd conectado (BFF + JWT)	28
17.Despliegue en AWS	30
18.Repositorios públicos de GitHub	31
19.Conclusiones y trabajo futuro	31
Referencias	32
Anexos	32

1. Introducción y resumen ejecutivo

FinZen constituye una aplicación web de finanzas personales, de naturaleza 100 % digital, con un asistente de IA que responde preguntas en lenguaje natural sobre las propias transacciones del usuario (por ejemplo: “¿en qué categoría gasto más?”). El sistema permite registrar ingresos y gastos por categoría, visualizar un dashboard con el balance y un gráfico de gastos, y consultar a un asistente basado en **RAG** (Retrieval-Augmented Generation) sobre embeddings.

El backend está compuesto por **4 microservicios en Python (FastAPI)**, cada uno con su **propia base de datos SQLite**, empaquetados en **contenedores Docker** y publicados detrás de **Nginx** como reverse proxy. El frontend es una aplicación **Next.js** que consume las APIs a través de un patrón **BFF** con autenticación **JWT**. Toda la solución se despliega sobre **una instancia EC2 en AWS** (región us-east-1) con **IP elástica**.

Alcance y objetivos

- Diseñar y documentar la propuesta (Parte A), el diseño del MVP (Parte B) y su implementación en contenedores (Parte C).
- Aplicar buenas prácticas de microservicios, contenedores y seguridad (JWT, contraseñas con hash, CORS restringido, usuario no root en Docker).
- Implementar un asistente RAG con embeddings, búsqueda híbrida y guardrails anti-alucinación.
- Desplegar en AWS EC2 y evidenciar el uso de las APIs con Postman.

Repositorios

- Backend: <https://github.com/UTEC-AII/finzen-app>
- Frontend: <https://github.com/UTEC-AII/finzen-webui>

Parte A: Análisis (QUÉ)

2. Empresa y producto propuesto

- **Empresa:** Nueva Startup Fintech (empresa ficticia).
- **Producto/servicio 100 % digital:** FinZen — Finanzas personales con asistente de IA.

FinZen permite registrar ingresos, sueldo y gastos diarios y, posteriormente, formular preguntas en lenguaje natural sobre la propia información financiera, mediante un motor de búsqueda semántica basado en *embeddings*.

3. Benchmark — Comparación con la competencia

Producto		Tipos de clientes	Funcionalidades	Propuesta de valor
PocketGuard		Personas que evitan sobregiros	Cálculo de “dinero disponible”, límites por categoría, vínculo bancario	Simplicidad: cuánto puedes gastar “hoy”
Copilot	Mo- ney	Usuarios Apple	Categorización con ML, detección de suscripciones, asistente conversacional	IA que aprende del usuario y reduce el trabajo manual
Monarch	Mo- ney	Individuos y parejas	Vínculo a miles de instituciones, patrimonio, metas en pareja, asistente IA	Todo-en-uno colaborativo con asistente IA

Cuadro 1: Cuadro comparativo del benchmark.

A partir de este análisis, FinZen adopta la simplicidad de PocketGuard y la interacción en lenguaje natural de Copilot Money y Monarch Money, con una diferencia sustantiva: no exige vincular cuentas bancarias. El usuario registra sus movimientos de forma manual y formula consultas en lenguaje natural sobre ellos.

4. Alcance funcional de FinZen (100 % digital)

Tipos de clientes / usuarios:

- Personas asalariadas con sueldo fijo mensual.
- Freelancers / independientes con ingresos variables.
- Estudiantes o jóvenes con mesada o ingresos ocasionales.

Funcionalidades (Historias de Usuario) — backlog priorizado:

#	Historia de Usuario	Prioridad	Pts	Depende de	Sprint
HU1	Registro y Login de usuario	Alta	3	—	Semana 1
HU2	Registrar Ingreso / Sueldo	Alta	3	HU1	Semana 1–2
HU3	Registrar Gasto por categoría	Alta	3	HU1	Semana 1–2
HU4	Listado y detalle de Ingresos	Media	2	HU2	Semana 2
HU5	Listado y detalle de Gastos	Media	2	HU3	Semana 2
HU6	Editar Perfil de Usuario	Baja	1	HU1	Semana 2
HU7	Consulta Inteligente (IA / embeddings)	Alta	5	HU2, HU3	Semana 3
HU8	Dashboard con resumen financiero	Alta	3	HU2, HU3, HU7	Semana 3–4

Cuadro 2: Backlog priorizado del MVP (total: 22 story points).

5. Propuesta de valor

A diferencia de PocketGuard, cuyo análisis se limita a estimar el gasto disponible del día, y de Copilot Money y Monarch Money, que condicionan el funcionamiento de su asistente a la vinculación de cuentas bancarias, **FinZen** propone un modelo de menor fricción: el usuario registra sus movimientos de forma manual y formula consultas en lenguaje natural, resueltas mediante recuperación semántica con *embeddings* (`text-embedding-3-large`). Esta propuesta no depende de integraciones bancarias ni de categorización manual.

6. Historias de Usuario y criterios de aceptación (Gherkin/BDD)

6.1. HU1 (MVP): Registro y Login

Como persona sin cuenta, **quiero** crear una cuenta e iniciar sesión con correo y contraseña, **para** acceder de forma segura a mi información.

Criterios de aceptación:

- **GIVEN** el usuario no tiene cuenta, **WHEN** completa nombre, correo, contraseña, moneda y país/zona horaria, **AND** presiona “Registrarse”, **THEN** se crea su perfil y accede a su dashboard.
- **GIVEN** el correo ya existe en `users.db`, **WHEN** intenta registrarse, **THEN** se muestra “correo ya registrado” y no se crea una cuenta duplicada.
- **GIVEN** una cuenta existente, **WHEN** ingresa credenciales correctas, **THEN** accede con sesión iniciada.
- **GIVEN** contraseña incorrecta, **WHEN** presiona “Entrar”, **THEN** se muestra un error genérico (sin revelar si el correo existe).

Tareas técnicas: FrontEnd `/register` y `/login`; BackEnd `POST /users` (hash bcrypt) y `POST /auth/login` (JWT); tabla `users` en `users.db`.

6.2. HU2 (MVP): Registrar Ingreso / Sueldo

Como usuario con sesión, **quiero** registrar un ingreso con tipo, monto, moneda, fecha y descripción, **para** llevar un registro completo.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** registra el ingreso, **THEN** se guarda en `incomes.db` y se envía a vectorizar.
- **GIVEN** monto vacío o negativo, **WHEN** guarda, **THEN** se muestra error de validación y no se registra.
- **GIVEN** el ingreso guardado, **WHEN** falla el `ai-service`, **THEN** el ingreso queda igualmente registrado (no bloquea al usuario).

Tareas técnicas: ruta `/incomes/new`; `POST /incomes`; integración con `POST /vectorize`; tabla `incomes`.

6.3. HU3 (MVP): Registrar Gasto por categoría

Como usuario con sesión, **quiero** registrar un gasto por categoría, **para** conocer el destino de mis gastos.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** registra el gasto, **THEN** se guarda en `expenses.db` y se vectoriza.
- **GIVEN** sin categoría seleccionada, **WHEN** guarda, **THEN** el sistema exige seleccionar una categoría.

Tareas técnicas: ruta `/expenses/new`; POST `/expenses`; GET `/expenses/categories/list`; tabla `expenses`.

6.4. HU4 (MVP): Listado y detalle de Ingresos

Como usuario con ingresos, **quiero** ver el listado y detalle, **para** revisar de dónde proviene mi dinero.

Criterios de aceptación:

- **GIVEN** ingresos registrados, **WHEN** accede a “Mis Ingresos” y selecciona uno, **THEN** ve el listado (ordenado por fecha) y el detalle completo.
- **GIVEN** sin ingresos, **WHEN** accede, **THEN** ve un estado vacío que invita a registrar el primero.

Tareas técnicas: rutas `/incomes` y `/incomes/[id]`; GET `/incomes/{user_id}`, GET `/incomes/detail/{id}`, DELETE `/incomes/{id}`.

6.5. HU5 (MVP): Listado y detalle de Gastos

Como usuario con gastos, **quiero** ver, filtrar y detallar mis gastos, **para** entender en qué gasto mi dinero.

Criterios de aceptación:

- **GIVEN** gastos registrados, **WHEN** accede a “Mis Gastos” y filtra por categoría o fecha, **THEN** ve el listado filtrado y el detalle del gasto seleccionado.
- **GIVEN** un filtro sin coincidencias, **WHEN** aplica, **THEN** se muestra un mensaje de “sin resultados”.

Tareas técnicas: rutas `/expenses` y `/expenses/[id]` (con eliminar); GET `/expenses/{user_id}` (filtros), GET `/expenses/detail/{id}`, DELETE `/expenses/{id}`.

6.6. HU6 (MVP): Editar Perfil de Usuario

Como usuario, **quiero** editar nombre, moneda, país/zona horaria y meta de ahorro, **para** mantener actualizada mi información.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** edita y guarda, **THEN** se actualizan sus datos (también la clave de OpenAI para el asistente).
- **GIVEN** intenta modificar el correo, **WHEN** está deshabilitado, **THEN** el correo permanece sin cambios.

Tareas técnicas: ruta `/profile`; GET `/users/{id}`, PUT `/users/{id}`.

6.7. HU7 (MVP): Consulta Inteligente de finanzas (IA / embeddings)

Como usuario con movimientos, **quiero** preguntar en lenguaje natural a un asistente sobre mis finanzas, **para** entender mis hábitos sin analizar los datos manualmente.

Criterios de aceptación:

- **GIVEN** movimientos registrados, **WHEN** escribe una pregunta (ej. “¿en qué categoría gasto más?”), **THEN** el sistema vectoriza la pregunta con `text-embedding-3-large`, recupera los movimientos más relevantes (búsqueda híbrida: similitud coseno + FTS5) y genera una respuesta en lenguaje natural.
- **GIVEN** el usuario sin movimientos, **WHEN** pregunta, **THEN** el asistente indica que no hay datos suficientes.
- **GIVEN** una pregunta fuera de contexto, **WHEN** no hay transacciones relevantes, **THEN** el asistente responde que no puede ayudar, sin inventar datos (guardrail).

Tareas técnicas: ruta `/assistant`; ai-service: POST `/vectorize` (interno), POST `/query`, POST `/reindex`, GET/PUT/DELETE `/settings/openai-key`; tablas `vectorized_transactions` y `settings` en `vectors.db`; pre-filtrado por metadata (categoría, tipo y fechas según la zona horaria del usuario).

6.8. HU8 (MVP): Dashboard con resumen financiero

Como usuario, **quiero** ver un resumen al entrar, **para** tener una visión general de mi balance y gastos.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** accede al Dashboard, **THEN** ve su balance (ingresos – gastos) por moneda, un gráfico de gastos por categoría y accesos directos.
- **GIVEN** sin movimientos, **WHEN** accede, **THEN** el balance es 0 y se le invita a registrar su primer movimiento.

Tareas técnicas: ruta /dashboard; agregación de datos de GET /incomes/{user_id} y GET /expenses/{user_id}.

Parte B: Diseño del MVP (CÓMO)

7. Diseño de FrontEnd (Prototipo) — Next.js

El FrontEnd se construye en Next.js (App Router) + TypeScript + Tailwind CSS y corre como un **contenedor Docker** sobre la misma instancia EC2, consumiendo los microservicios. A partir de las 8 historias se definieron estas rutas:

Ruta (Next.js)	Vista	HU
/	Bienvenida / Landing	—
/login	Inicio de sesión	HU1
/register	Creación de cuenta (moneda y zona horaria)	HU1
/dashboard	Balance por moneda, gráfico, accesos directos	HU8
/incomes/new	Registrar ingreso	HU2
/incomes, /incomes/[id]	Listado y detalle de ingresos	HU4
/expenses/new	Registrar gasto	HU3
/expenses, /expenses/[id]	Listado (filtros) y detalle	HU5
/profile	Editar perfil (y clave de OpenAI)	HU6
/assistant	Chat de Consulta Inteligente	HU7

Cuadro 3: Rutas del prototipo (frontend).

El prototipo responsivo (`FinZen_mockup.html`) y el prompt usado en herramientas de IA están en `frontend-prompt/` del repositorio del frontend.

8. Diseño de BackEnd — FastAPI en contenedores Docker

El BackEnd se implementa con **FastAPI**, como **4 contenedores Docker independientes** orquestados mediante **Docker Compose**. Cada contenedor sigue el principio de responsabilidad única y mantiene su propia base de datos **SQLite** persistida en un volumen.

Microservicio	Puerto	BD	Métodos principales
user-service	8001	users.db	POST /users, POST /auth/login, GET /users/{id}, PUT /users/{id}
income-service	8002	incomes.db	POST /incomes, GET /incomes/{user_id}, GET /incomes/detail/{id}, DELETE /incomes/{id}
expense-service	8003	expenses.db	POST /expenses, GET /expenses/{user_id}, GET /expenses/detail/{id}, GET /expenses/categories/list, DELETE /expenses/{id}
ai-service	8004	vectors.db	POST /vectorize, POST /query, POST /reindex, GET/PUT/DELETE /settings/openai-key

Cuadro 4: Microservicios del backend.

Un contenedor **Nginx** (en el mismo `docker-compose.yml`) actúa como reverse proxy: expone el FrontEnd en `/` y enruta `/api/users`, `/api/incomes`, `/api/expenses` y `/api/ai` a los puertos 8001–8004, resolviéndolos por el **nombre del servicio** en la red interna.

8.1. Estructura de la base de datos (SQL — SQLite)

Las columnas van en inglés y los montos se guardan como `Numeric(12,2)` (Decimal), no como `Float`, para no perder precisión financiera.

```
// FinZen - Esquema de base de datos (DBML para dbdiagram.io)
// 4 microservicios, cada uno con su propia base SQLite.
```

```
// Las referencias son "logicas" (no hay FK fisica entre bases distintas).
```

```
Table users {
  id varchar [pk]
  name varchar [not null]
  email varchar [not null, unique]
  password_hash varchar [not null]
  preferred_currency varchar [default: 'PEN']
  timezone varchar [default: 'America/Lima']
  monthly_savings_goal numeric(12,2) [default: 0]
  created_at varchar [not null]
  Note: 'user-service - users.db'
}
```

```
Table incomes {
  id varchar [pk]
  user_id varchar [not null]
  type varchar [not null] // Sueldo | Freelance | Bono | Inversion | Otro
  amount numeric(12,2) [not null]
  currency varchar [not null]
  date date [not null]
  description varchar
  created_at varchar [not null]
  Note: 'income-service - incomes.db'

  indexes {
    (user_id, date) [name: 'idx_incomes_user_date']
    (user_id, type) [name: 'idx_incomes_user_type']
  }
}
```

```
Table expenses {
  id varchar [pk]
  user_id varchar [not null]
  category varchar [not null] // Alimentacion | Transporte | Vivienda |
  ...
  amount numeric(12,2) [not null]
  currency varchar [not null]
  date date [not null]
  description varchar
  created_at varchar [not null]
  Note: 'expense-service - expenses.db'

  indexes {
```

```
(user_id, date) [name: 'idx_expenses_user_date']
(user_id, category) [name: 'idx_expenses_user_category']
}
}

Table vectorized_transactions {
  id varchar [pk]
  user_id varchar [not null]
  record_type varchar [not null] // income | expense
  category varchar
  currency varchar
  amount numeric(12,2)
  date varchar
  description varchar
  text varchar [not null] // frase vectorizada (incluye moneda)
  embedding text [not null] // vector de 3072 dims (JSON)
  embedding_model varchar // text-embedding-3-large | local-hash
  created_at varchar [not null]
  Note: 'ai-service - vectors.db'

  indexes {
    (user_id, record_type) [name: 'idx_vectors_user_type']
  }
}

Table settings {
  key varchar [pk]
  value text [not null]
  Note: 'ai-service - vectors.db (clave de OpenAI, etc.)'
}

// Relaciones logicas (FK por user_id entre microservicios)
Ref: incomes.user_id > users.id
Ref: expenses.user_id > users.id
Ref: vectorized_transactions.user_id > users.id
```

Código 1: Esquema de las bases de datos (DBML — dbdiagram.io).

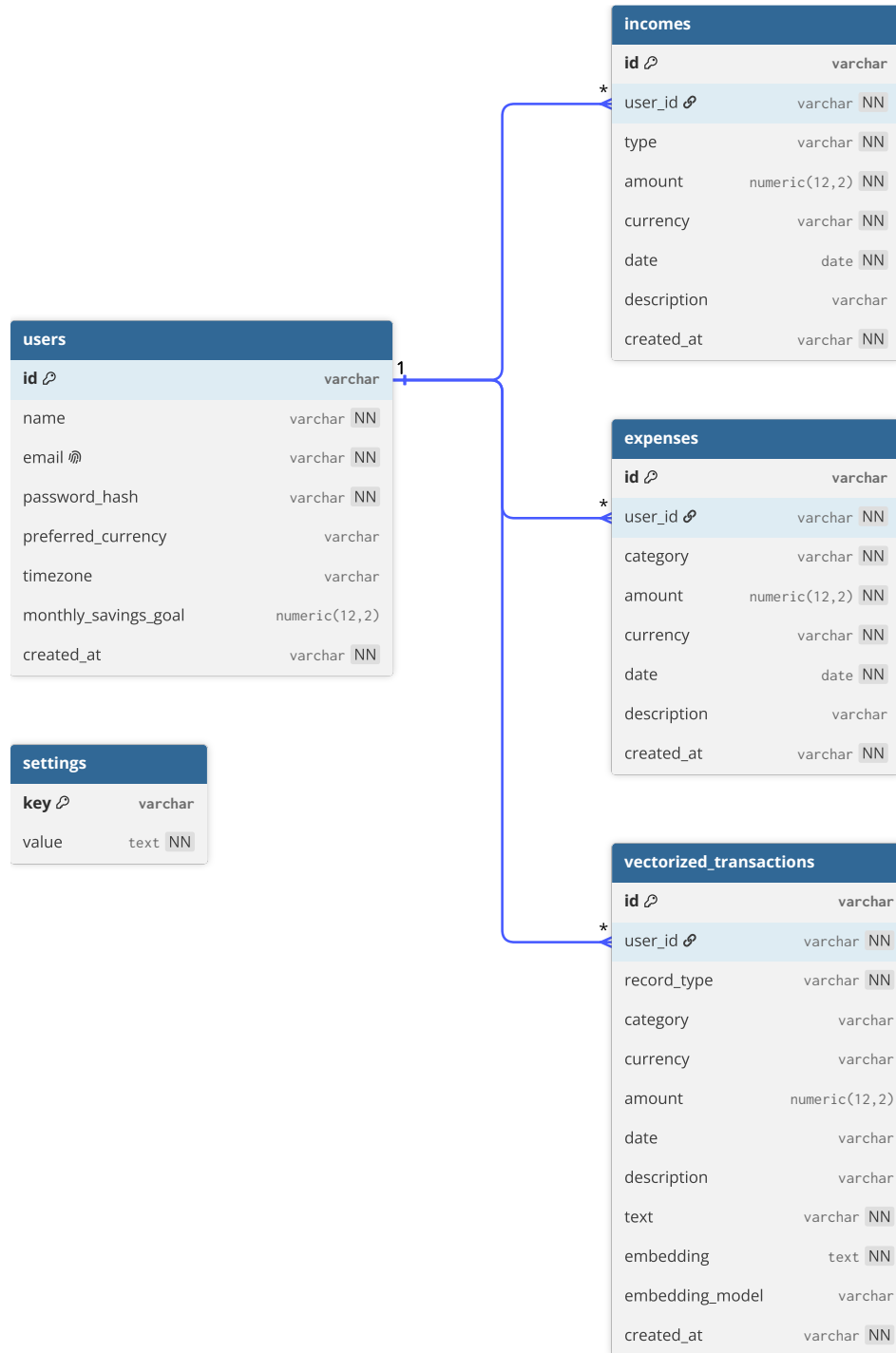


Figura 1: Diagrama Entidad-Relación de las bases de datos (elaborado con dbdiagram.io).

8.2. Selección del motor de base de datos: SQLite frente a PostgreSQL

PostgreSQL, preferentemente en Amazon RDS, constituye la alternativa convencional para entornos de producción con múltiples instancias, dado que permite compartir el estado a través de la red. No obstante, la solución propuesta se ejecuta sobre **una única instancia EC2**, por lo que no existe el problema de compartir estado entre servidores y SQLite resulta suficiente. Además, SQLite no requiere un servidor de base de datos independiente, simplifica el despliegue y no genera costo adicional. La migración a PostgreSQL se justificaría si la solución escalara a cientos o miles de usuarios concurrentes.

9. Empaquetado con Docker

Cada microservicio y el FrontEnd se empaquetan como imágenes Docker independientes. Los Dockerfiles parten de imágenes base oficiales de **Docker Hub** (python:3-slim y node:20-alpine).

```
FROM python:3-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN addgroup --system appgroup && adduser --system --ingroup appgroup
    appuser \
    && mkdir -p /app/data && chown -R appuser:appgroup /app
USER appuser
EXPOSE 8001
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://
    localhost:8001/health')" || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Código 2: Dockerfile típico de un microservicio FastAPI.

```
services:
  user-service: { build: ./user-service, ports: ["8001:8001"],
    volumes: ["users_data:/app/data"] }
  income-service: { build: ./income-service, ports: ["8002:8002"],
    volumes: ["incomes_data:/app/data"] }
  expense-service: { build: ./expense-service, ports: ["8003:8003"],
    volumes: ["expenses_data:/app/data"] }
```

```

ai-service:      { build: ./ai-service,      ports: ["8004:8004"],
                  volumes: ["vectors_data:/app/data"] }
nginx:          { image: nginx:alpine,      ports: ["80:80"],
                  depends_on: [user-service, income-service, expense-service, ai-service]
                  }
volumes: { users_data: {}, incomes_data: {}, expenses_data: {},
          vectors_data: {} }

```

Código 3: docker-compose.yml (resumen).

Persistencia: los contenedores son efímeros; cada servicio monta un **volumen** sobre /app/data (EBS), de modo que el archivo .db sobrevive a reinicios o redeploy.

10. Diagrama de Arquitectura de Solución en AWS

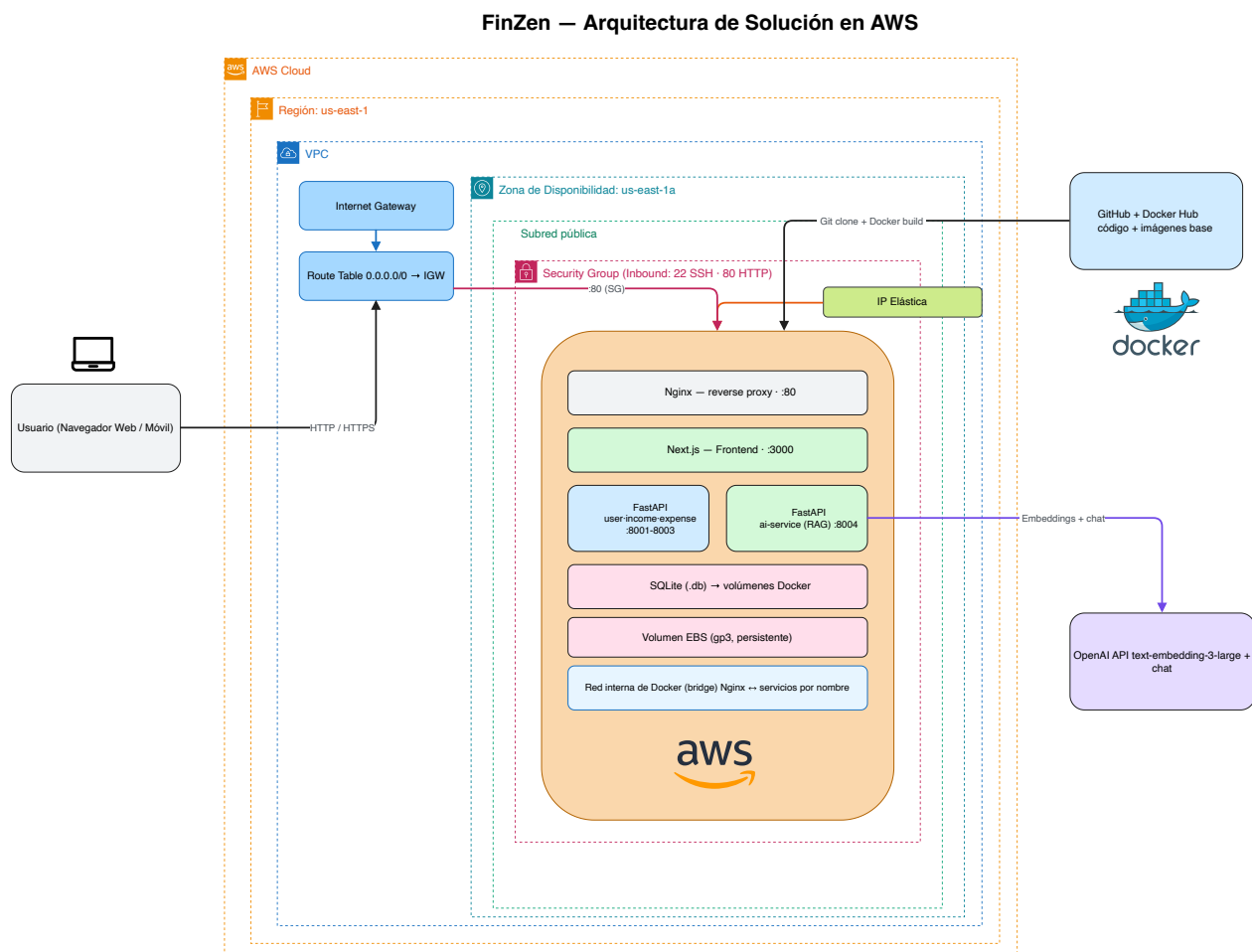


Figura 2: Arquitectura de FinZen en AWS (una instancia EC2 con IP elástica).

La solución emplea **una única instancia EC2** como servicio de cómputo y prescinde de Application Load Balancer, Auto Scaling, CloudFront, S3 y CloudWatch, en concordancia con el alcance mínimo del proyecto:

- Una instancia **EC2 t3.micro** en **us-east-1** con **IP elástica**: punto de entrada único.
- Sobre ella corre **Docker Engine** y, vía **Docker Compose**, 6 contenedores (Nginx, Next.js y los 4 FastAPI), comunicados por la red interna; Nginx publica todo en el puerto 80.
- El **User Data** clona los repositorios, construye las imágenes (**docker build**) y levanta los contenedores.
- Las bases SQLite viven en **volúmenes Docker** sobre el volumen **EBS**. El **ai-service** llama a la API de OpenAI.
- La **OPENAI_API_KEY** se inyecta como variable de entorno, sin Secrets Manager (arquitectura mínima solicitada).
- **IP fija**: se asigna una **IP elástica** para que la dirección no cambie al reiniciar la instancia.

10.1. Red en AWS (VPC, Internet Gateway, subred y Security Group)

La solución se despliega en la **VPC por defecto**, que ya incluye:

- **Internet Gateway (IGW)** adjunto a la VPC: conecta la VPC con internet.
- **Route Table** con la ruta **0.0.0.0/0 → Internet Gateway**.
- **Subredes públicas** en una **Zona de Disponibilidad** de **us-east-1**.
- **Security Group** (firewall virtual de la instancia) con reglas de entrada: **22** (SSH, para EC2 Instance Connect) y **80** (HTTP, por donde entra todo el tráfico web); y salida *All traffic* para **docker pull**, **git clone** y la API de OpenAI.

El **Internet Gateway** proporciona conectividad a la instancia en ambos sentidos: **entrante** (los usuarios acceden por el puerto 80) y **saliente** (descarga de imágenes de Docker Hub,

clonado desde GitHub y llamadas a OpenAI). No se emplea **NAT Gateway**, ya que este sólo aplica a subredes privadas y añade costo; dado que se trata de una subred pública con IP elástica, el Internet Gateway constituye la alternativa técnica y económicamente adecuada.

11. Selección de región AWS

Región elegida: `us-east-1` (Norte de Virginia).

Justificación: se seleccionó por ser la región con mayor disponibilidad de servicios y con los costos más competitivos de EC2 y EBS, factor determinante para un proyecto académico. Asimismo, dado que la solución depende de una API externa (OpenAI) con buena conectividad desde Estados Unidos, la latencia adicional respecto de una región más cercana a Perú (`sa-east-1`) resulta marginal en comparación con el ahorro de costos.

12. Costos mensuales estimados en AWS

Estimación con la Calculadora de AWS (<https://calculator.aws>) para una sola instancia EC2:

Servicio	Configuración	Costo/mes (USD)
EC2	1x t3.micro on-demand, Linux, 24/7	\$7.59
EBS	gp3, 20 GB (imágenes Docker, SQLite y código)	\$1.60
IP elástica (IPv4)	1 dirección elástica (~\$0.005/h, se cobre o no)	\$3.60
Transferencia de datos	~5 GB/mes	\$0.45
TOTAL AWS (estimado)		~\$13.24 / mes

Cuadro 5: Costos mensuales estimados.

Consideración de costos: desde febrero de 2024, AWS factura **toda dirección IPv4 pública** (~\$0.005 por hora), con independencia de que esté asociada o no a una instancia; en consecuencia, la **IP elástica** representa ~\$3.60/mes. No obstante, al operar con una única instancia se evita el costo de un Application Load Balancer (~\$16/mes) y de una segunda instancia EC2.

El consumo de la API de OpenAI (embeddings y modelo de chat) se factura de forma independiente, fuera de AWS.

12.1. Costos de OpenAI (aparte de AWS)

Precios oficiales de la API (fuente: <https://platform.openai.com/docs/pricing>), por 1 millón de tokens (plan Standard).

Modelo de embeddings	Precio / 1M tokens
text-embedding-3-large (usado por FinZen)	\$0.13
text-embedding-3-small	\$0.02

Cuadro 6: Precios de embeddings de OpenAI.

Modelo de chat	Entrada / 1M	Salida / 1M
gpt-4o-mini (usado por FinZen)	\$0.15	\$0.60
gpt-4.1-nano	\$0.10	\$0.40
gpt-4.1-mini	\$0.40	\$1.60
gpt-4o	\$2.50	\$10.00
gpt-3.5-turbo	\$0.50	\$1.50

Cuadro 7: Precios de modelos de chat de OpenAI.

Para el volumen estimado del proyecto (1000 movimientos vectorizados y 200 consultas mensuales), el costo de IA asciende a aproximadamente **\$0.04/mes** (embeddings `text-embedding-3-large` y chat `gpt-4o-mini`), un valor marginal en relación con el costo de AWS. Cabe señalar que existen modalidades más económicas: procesamiento por lotes (*Batch*, $\approx 50\%$ menor) y caché de entrada ($\approx 90\%$ menor en tokens repetidos).

Parte C: Implementación del MVP

13. Catálogo de APIs

Todos los servicios exponen respuestas en formato JSON y documentan su interfaz mediante OpenAPI/Swagger (`/docs`). Nginx publica cada microservicio bajo `/api/*`. Los endpoints protegidos requieren `Authorization: Bearer <token>`.

ServicioMétodo		Ruta pública (Nginx)	Auth	Descripción
user	POST	/api/users/users	No	Registrar usuario
user	POST	/api/users/auth/login	No	Login (emite JWT)
user	GET	/api/users/users/{id}	Sí	Perfil
user	PUT	/api/users/users/{id}	Sí	Actualizar perfil
income	POST	/api/incomes/incomes	Sí	Registrar ingreso
income	GET	/api/incomes/incomes/{user_id}	Sí	Listar ingresos
income	GET	/api/incomes/incomes/detail/{id}	Sí	Detalle
income	DELETE	/api/incomes/incomes/{id}	Sí	Eliminar
expense	POST	/api/expenses/expenses	Sí	Registrar gasto
expense	GET	/api/expenses/expenses/{user_id}	Sí	Listar (filtros)
expense	GET	/api/expenses/expenses/detail/{id}	Sí	Detalle
expense	GET	/api/expenses/expenses/categories/list	Sí	Categorías
expense	DELETE	/api/expenses/expenses/{id}	Sí	Eliminar
ai	POST	/api/ai/query	Sí	Consulta en lenguaje natural (RAG)
ai	POST	/api/ai/reindex	Sí	Re-indexar movimientos
ai	GET/PUT/DELETE	/api/ai/settings/openai-key	Sí	Clave de OpenAI

14. Implementación del BackEnd

El backend se compone de 4 microservicios en Python (FastAPI), cada uno empaquetado como contenedor Docker con su propia base SQLite persistida en un volumen y orquestados mediante Docker Compose. Los endpoints protegidos emplean JWT. Cada servicio expone `/health` e implementa prácticas de calidad y seguridad: CORS restringido, validación con Pydantic, ejecución con usuario no privilegiado y `HEALTHCHECK` en Docker.

15. Evidencia de uso con Postman

La colección importable, junto con los entornos Local y AWS y la guía de evidencias, se encuentra en `postman/` del repositorio del backend. A continuación se presentan las capturas de las peticiones ejecutadas (registro, inicio de sesión, registro de ingresos y gastos, filtrado, las tres consultas al asistente y la reindexación), con sus códigos de estado reales.

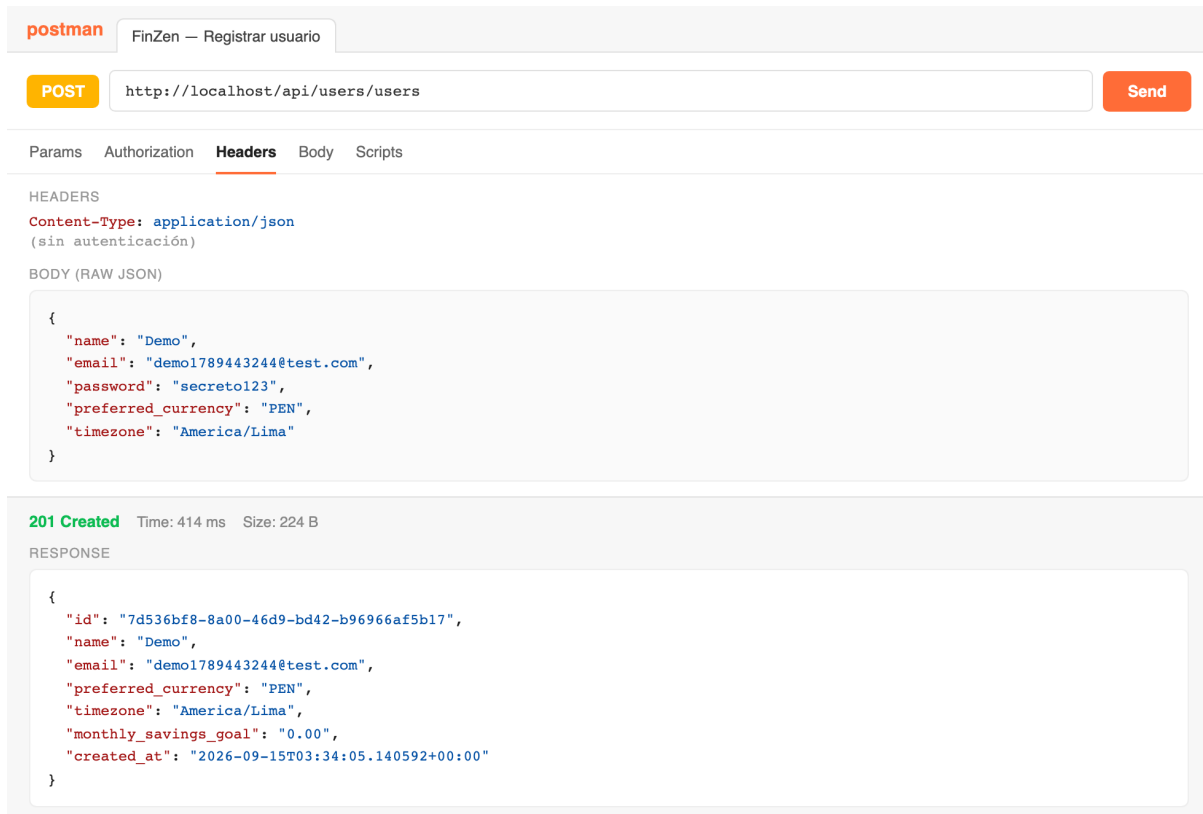


Figura 3: Registro de usuario (POST `/api/users/users`, 201 Created).

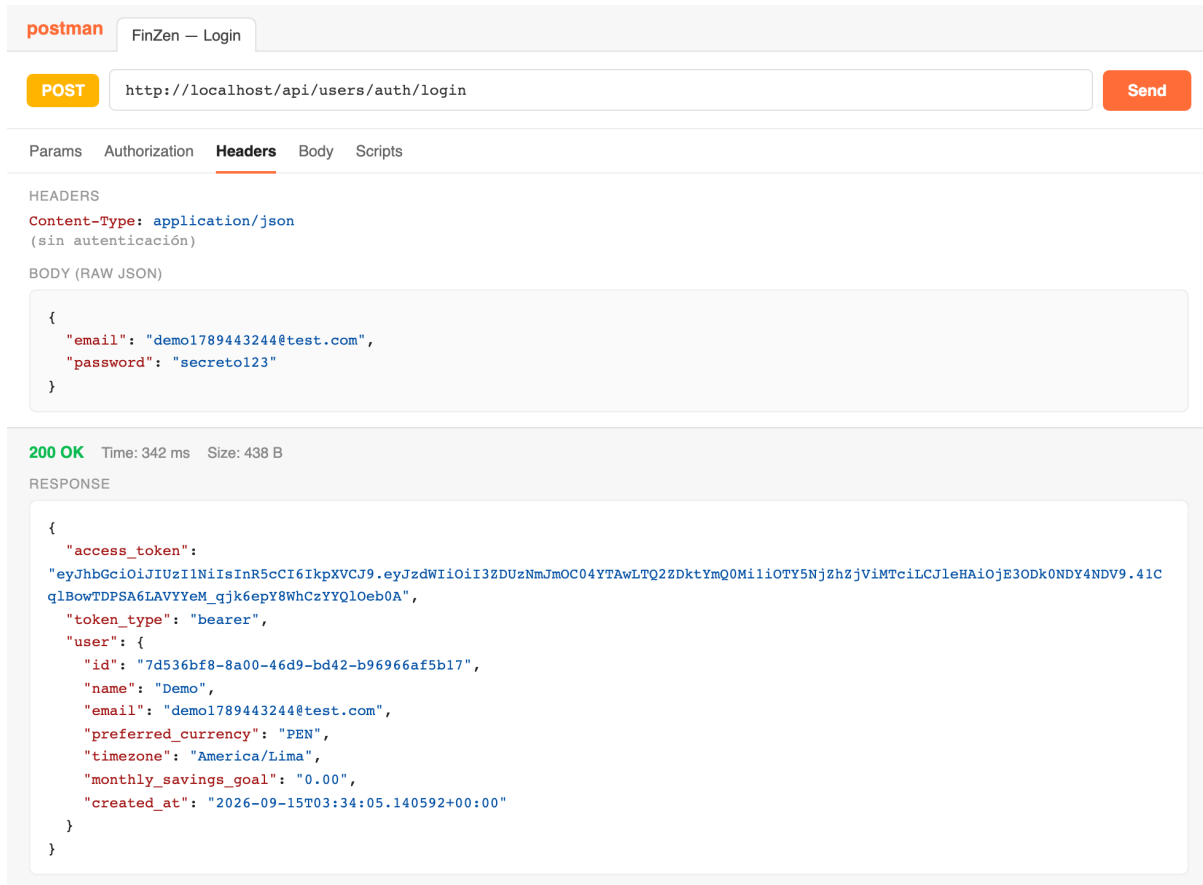


Figura 4: Login (POST /api/users/auth/login, 200 OK) devolviendo el access_token.

The image shows a Postman interface for a REST client. At the top, the title bar says "postman" and "FinZen — Registrar ingreso". Below this, the method is set to "POST" and the URL is "http://localhost/api/incomes/incomes". A "Send" button is on the right. The "Headers" tab is selected, showing "Content-Type: application/json" and "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...". The "Body" tab is also visible, showing a raw JSON body:

```
{  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",  "type": "Sueldo",  "amount": "3500.50",  "currency": "PEN",  "date": "2026-09-01",  "description": "Sueldo de setiembre"} 
```

 Below the body, the response status is "201 Created" with a time of 5 ms and size of 250 B. The "RESPONSE" tab is selected, showing the JSON response:

```
{  "id": "448569ca-050d-4c01-9bc5-563fc54395be",  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",  "type": "Sueldo",  "amount": "3500.50",  "currency": "PEN",  "date": "2026-09-01",  "description": "Sueldo de setiembre",  "created_at": "2026-09-15T03:34:05.490734+00:00"} 
```

Figura 5: Registrar ingreso (POST /api/incomes/incomes, 201).

The image shows a Postman interface for a REST client. At the top, the title bar says "postman" and "FinZen — Registrar gasto (PEN)". Below this, the method is set to "POST" and the URL is "http://localhost/api/expenses/expenses". A "Send" button is on the right. The "Headers" tab is selected, showing "Content-Type: application/json" and "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...". The "Body" tab is also visible, showing a raw JSON body:

```
{  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",  "category": "Alimentacion",  "amount": "450.50",  "currency": "PEN",  "date": "2026-09-05",  "description": "Supermercado"}
```

. Below the body, a status bar indicates "201 Created", "Time: 5 ms", and "Size: 252 B". The "RESPONSE" tab is selected, showing a JSON response:

```
{  "id": "4af9ee3e-e163-4fb6-9e2d-acb1f58e029f",  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",  "category": "Alimentacion",  "amount": "450.50",  "currency": "PEN",  "date": "2026-09-05",  "description": "Supermercado",  "created_at": "2026-09-15T03:34:05.496371+00:00"}
```

Figura 6: Registrar gasto en PEN (POST /api/expenses/expenses, 201).

The screenshot shows a Postman interface for a REST client. The top bar indicates the environment is 'FinZen' and the active tab is 'Registrar gasto (USD)'. The request method is 'POST' and the URL is 'http://localhost/api/expenses/expenses'. The 'Send' button is visible. Below the URL bar, the 'Headers' tab is selected, showing 'Content-Type: application/json' and 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...'. The 'Body' tab is also visible, showing a raw JSON payload. The response status is '201 Created' with a time of 3 ms and a size of 249 B. The response body is a JSON object containing the details of the created expense, including an 'id', 'user_id', 'category', 'amount', 'currency', 'date', 'description', and 'created_at'.

postman FinZen — Registrar gasto (USD)

POST http://localhost/api/expenses/expenses Send

Params Authorization **Headers** Body Scripts

HEADERS

Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

BODY (RAW JSON)

```
{
  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",
  "category": "Entretenimiento",
  "amount": "20.00",
  "currency": "USD",
  "date": "2026-09-06",
  "description": "Netflix"
}
```

201 Created Time: 3 ms Size: 249 B

RESPONSE

```
{
  "id": "4e0de04b-b88c-4fc3-a5c4-de384bfa17ad",
  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",
  "category": "Entretenimiento",
  "amount": "20.00",
  "currency": "USD",
  "date": "2026-09-06",
  "description": "Netflix",
  "created_at": "2026-09-15T03:34:05.501564+00:00"
}
```

Figura 7: Registrar gasto en USD (201).

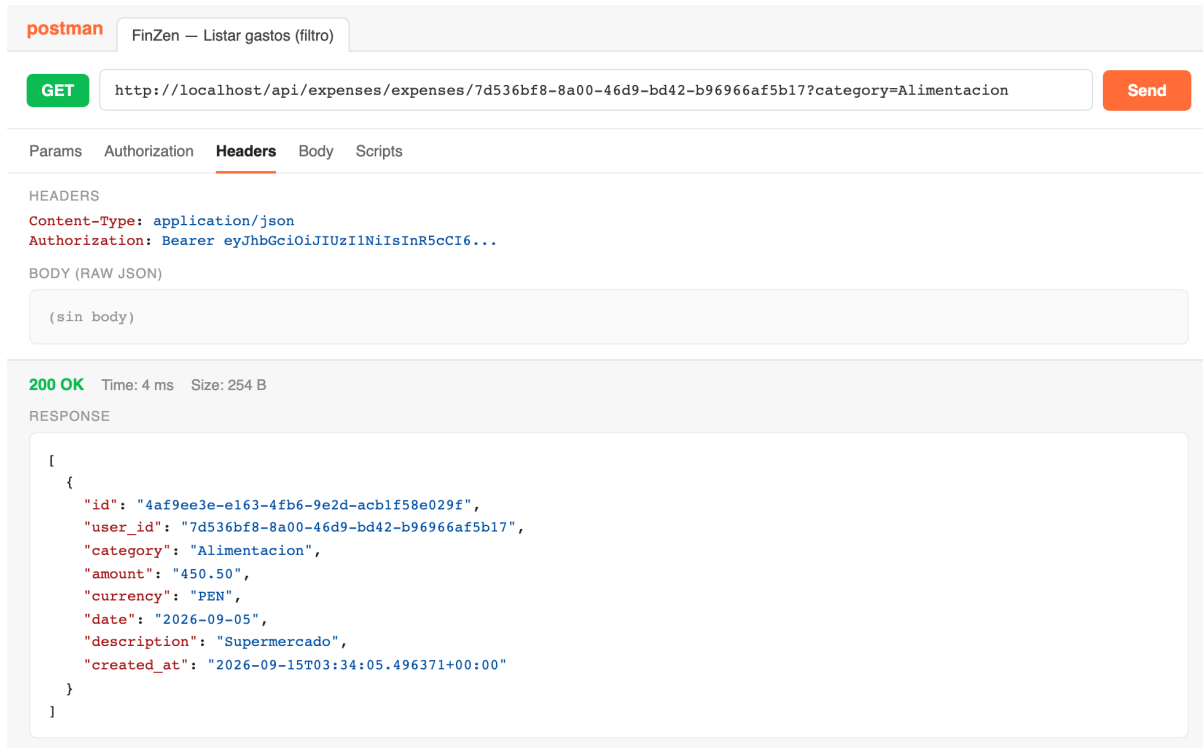


Figura 8: Listar gastos con filtro por categoría (GET `/api/expenses/expenses/{user_id}?category=Alimentacion`, 200).

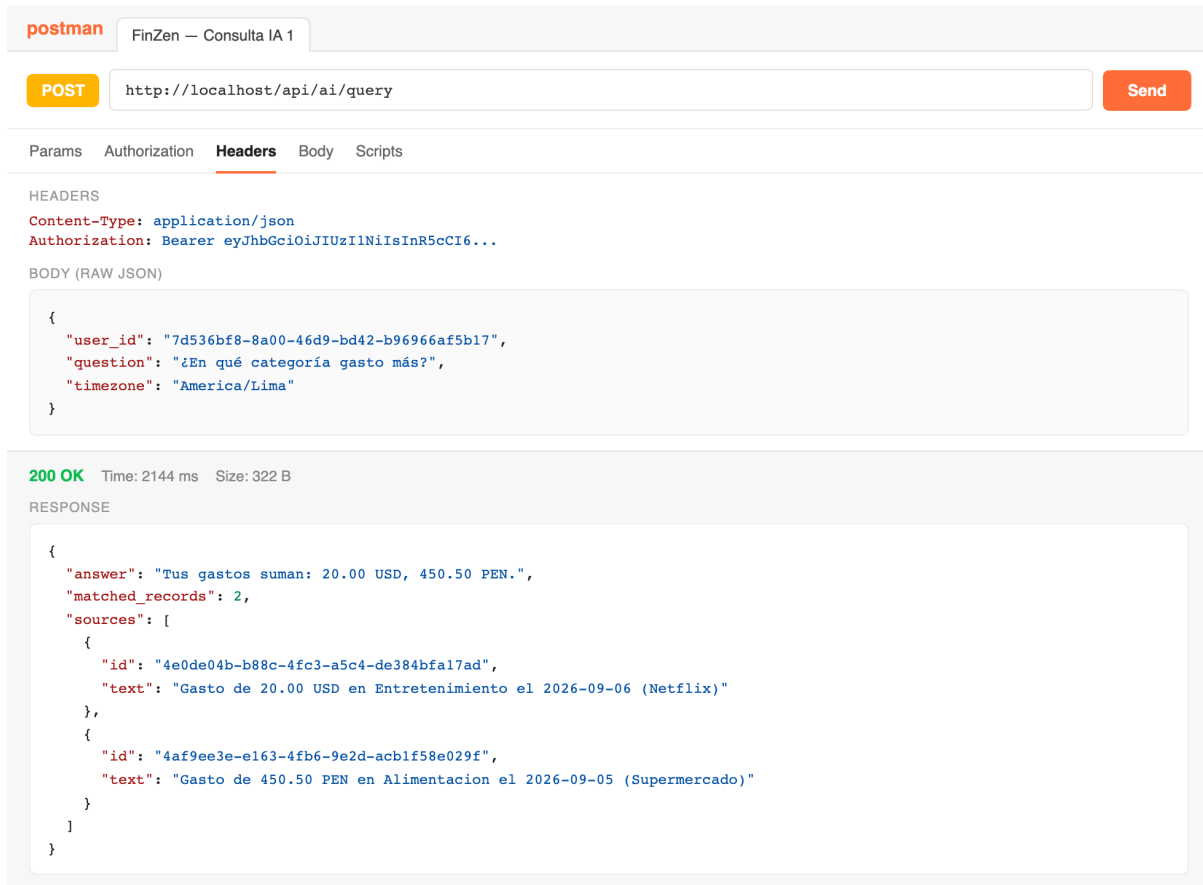
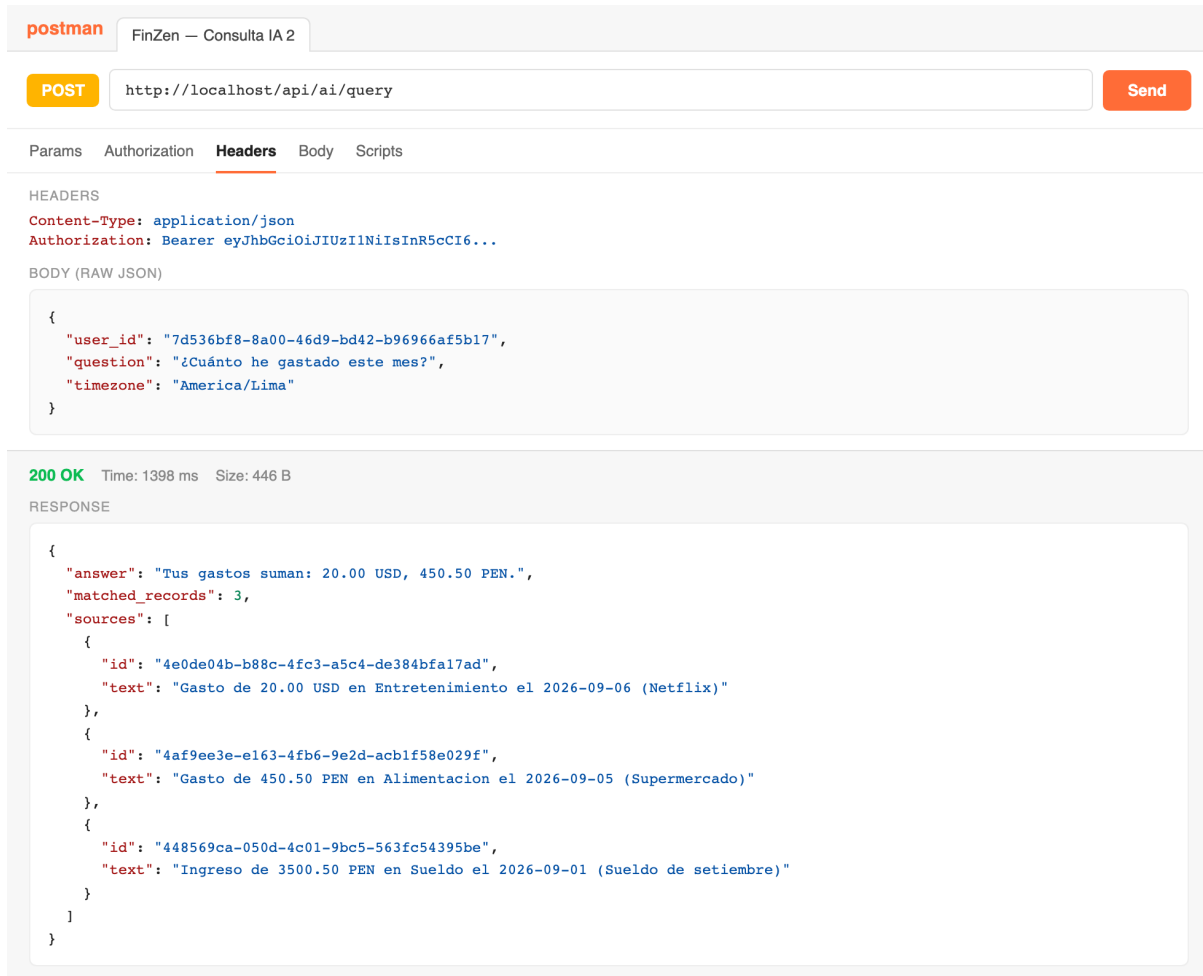


Figura 9: Consulta IA 1: “¿en qué categoría gasto más?” (POST /api/ai/query, 200).



postman FinZen — Consulta IA 2

POST http://localhost/api/ai/query Send

Params Authorization **Headers** Body Scripts

HEADERS

Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

BODY (RAW JSON)

```
{
  "user_id": "7d536bf8-8a00-46d9-bd42-b96966af5b17",
  "question": "¿Cuánto he gastado este mes?",
  "timezone": "America/Lima"
}
```

200 OK Time: 1398 ms Size: 446 B

RESPONSE

```
{
  "answer": "Tus gastos suman: 20.00 USD, 450.50 PEN.",
  "matched_records": 3,
  "sources": [
    {
      "id": "4e0de04b-b88c-4fc3-a5c4-de384bfa17ad",
      "text": "Gasto de 20.00 USD en Entretenimiento el 2026-09-06 (Netflix)"
    },
    {
      "id": "4af9ee3e-e163-4fb6-9e2d-acb1f58e029f",
      "text": "Gasto de 450.50 PEN en Alimentacion el 2026-09-05 (Supermercado)"
    },
    {
      "id": "448569ca-050d-4c01-9bc5-563fc54395be",
      "text": "Ingreso de 3500.50 PEN en Sueldo el 2026-09-01 (Sueldo de setiembre)"
    }
  ]
}
```

Figura 10: Consulta IA 2: “¿cuánto he gastado este mes?” (200).

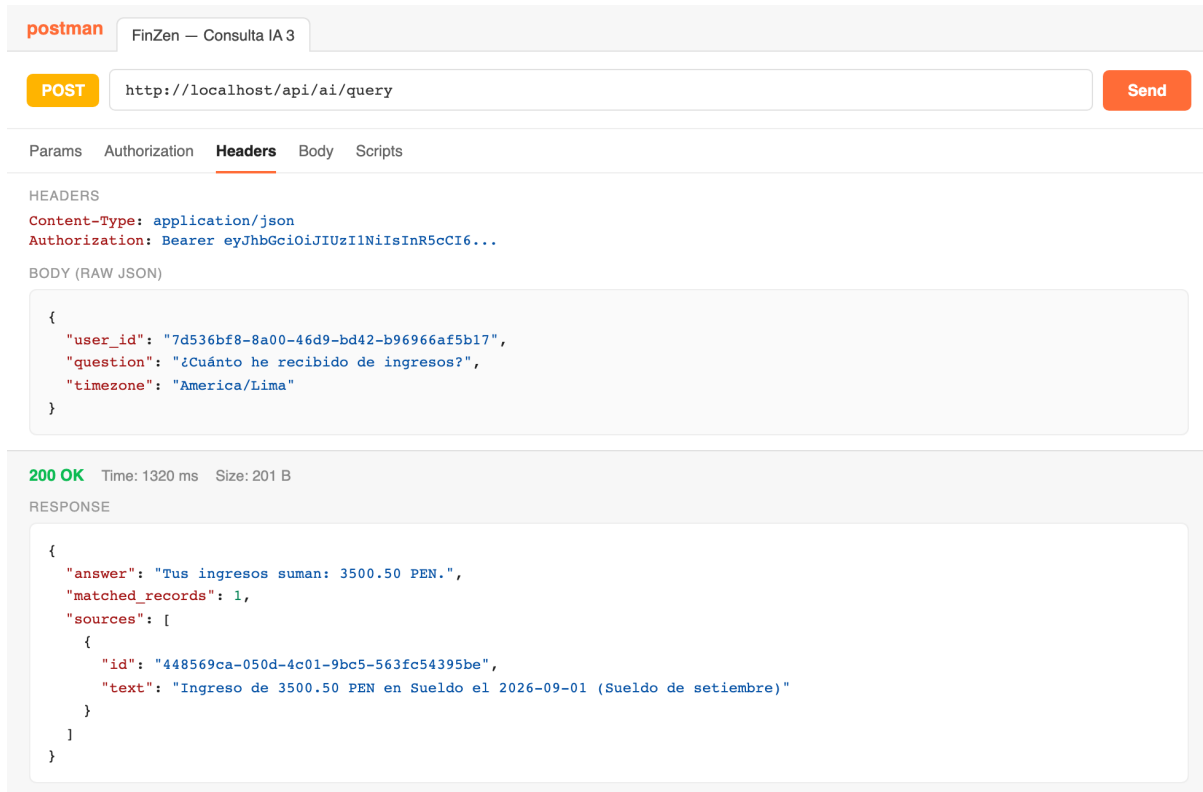


Figura 11: Consulta IA 3: “¿cuánto he recibido de ingresos?” (200).

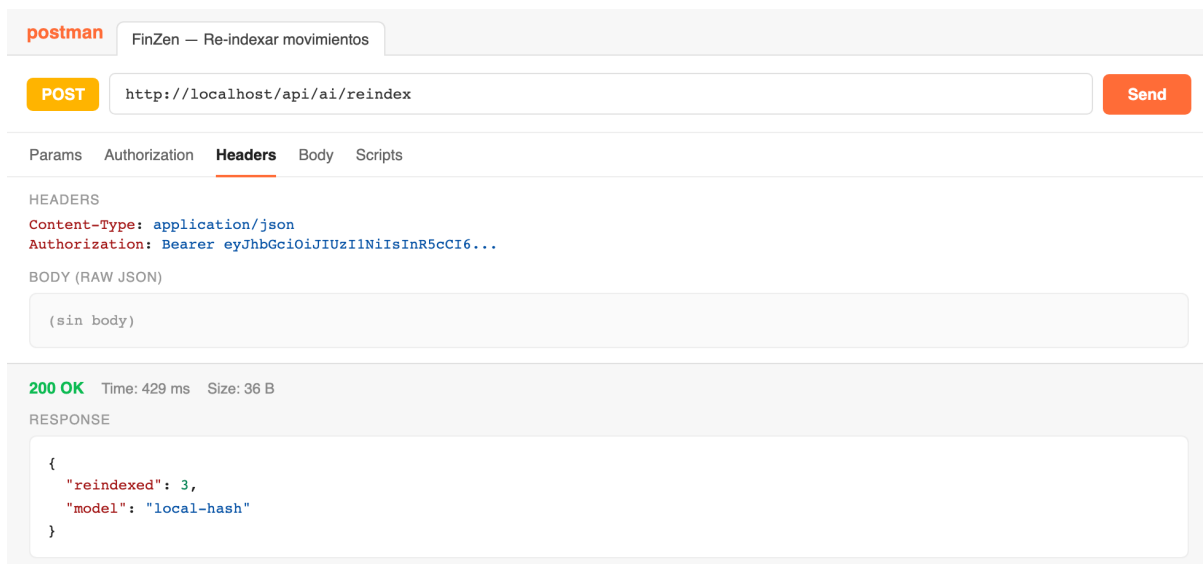


Figura 12: Re-indexar movimientos (POST /api/ai/reindex, 200).

16. FrontEnd conectado (BFF + JWT)

El frontend Next.js consume las APIs a través de un **BFF**: el JWT se guarda en una cookie `httpOnly` y el servidor de Next lo inyecta al reenviar la petición a Nginx.

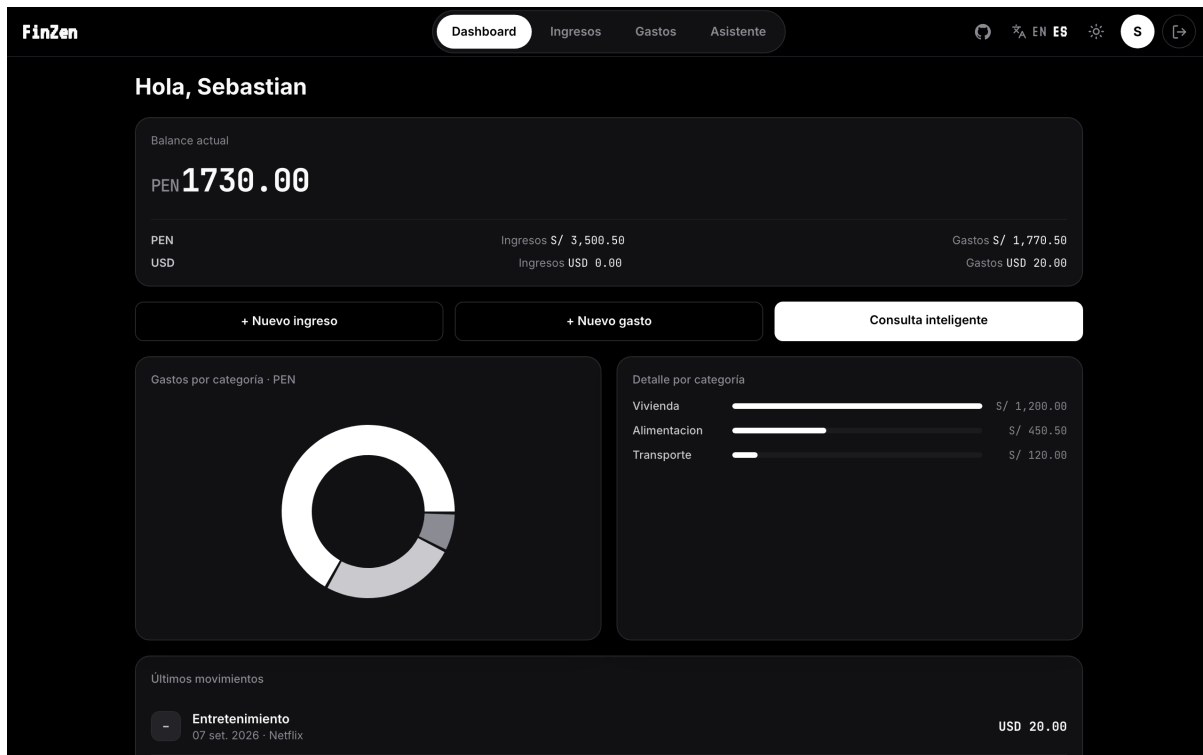


Figura 13: Dashboard: balance por moneda, gráfico de gastos por categoría y accesos directos.

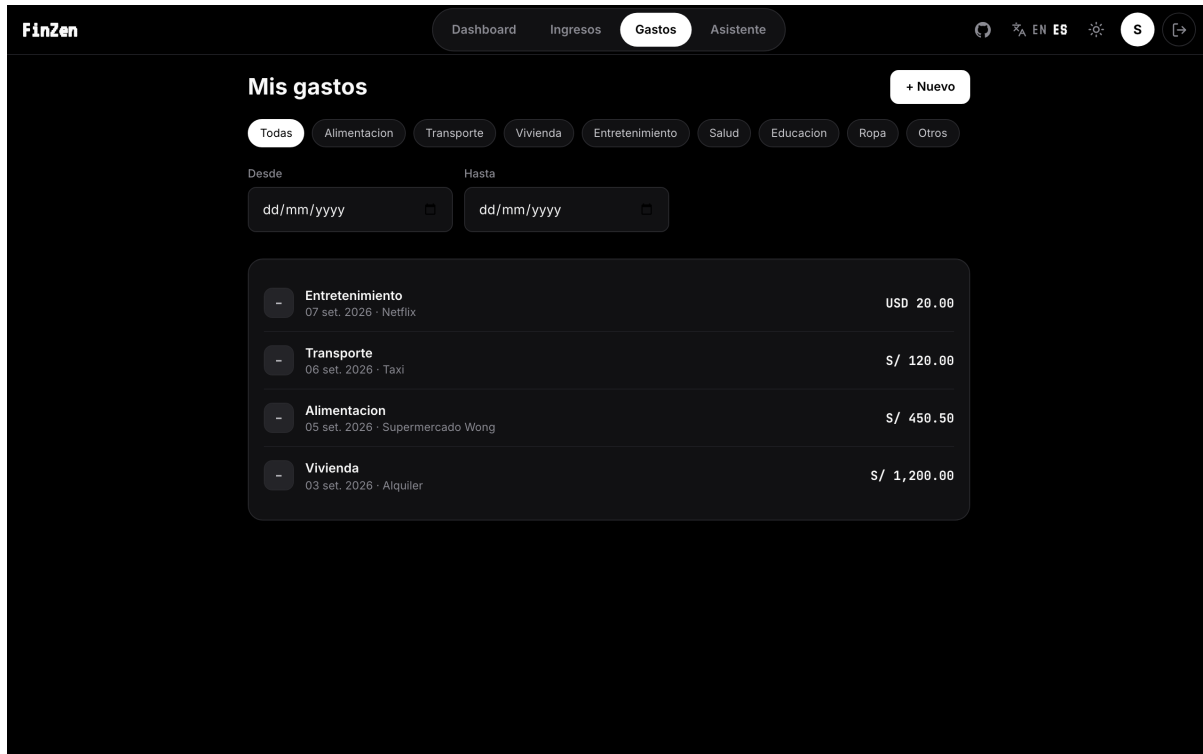


Figura 14: Listado de gastos con filtros por categoría y rango de fechas.

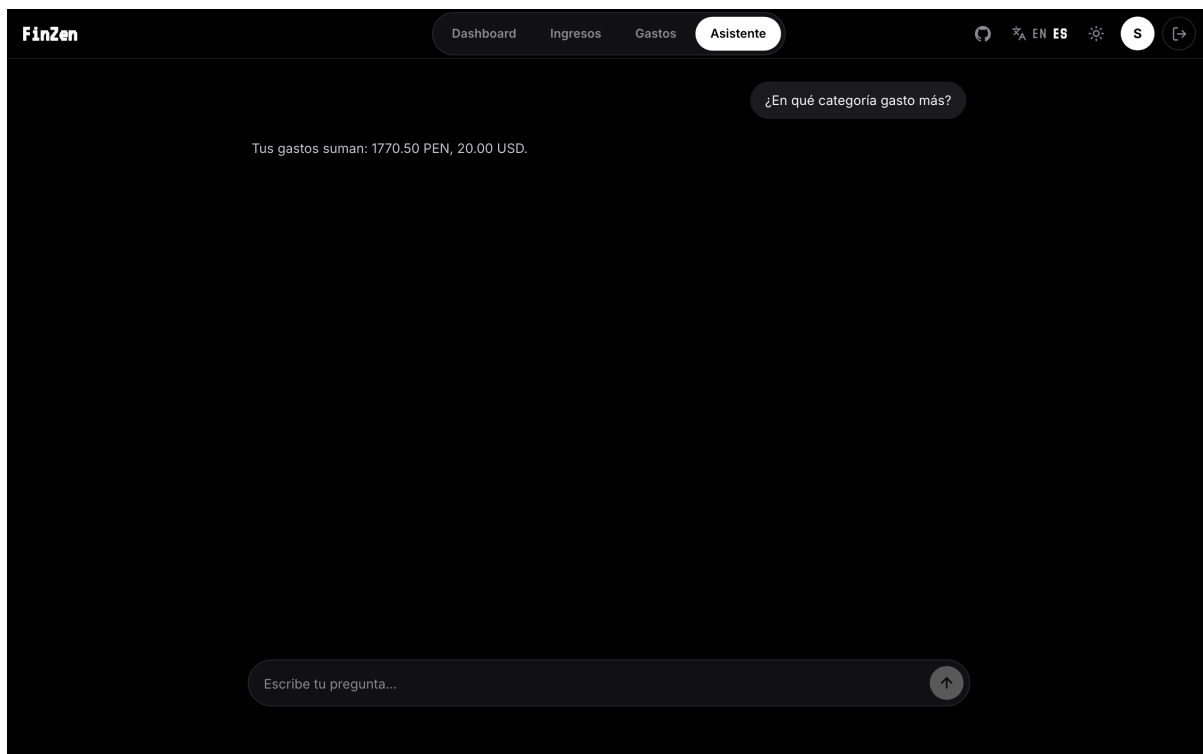


Figura 15: Asistente de IA respondiendo una consulta en lenguaje natural.

17. Despliegue en AWS

El despliegue se realiza sobre **una instancia EC2** con **IP elástica**, administrada mediante **EC2 Instance Connect** (sin gestión de llaves SSH). El script de *User Data* instala Docker, clona los repositorios y ejecuta `docker compose up -d -build`.

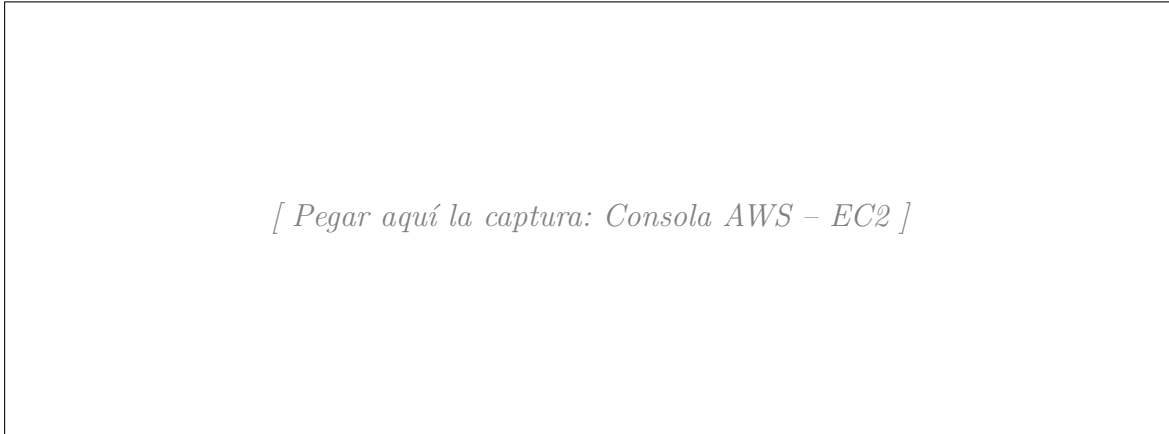
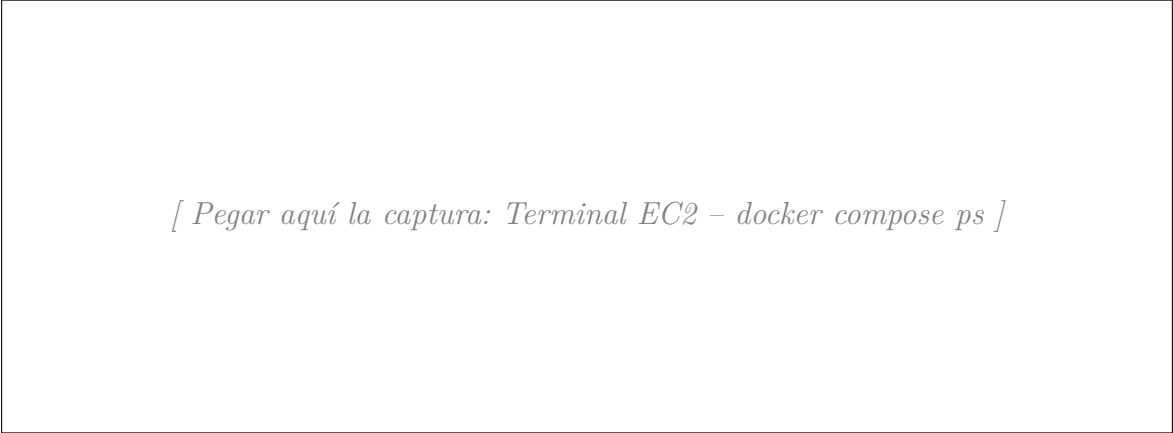


Figura 16: Instancia EC2 con IP elástica.

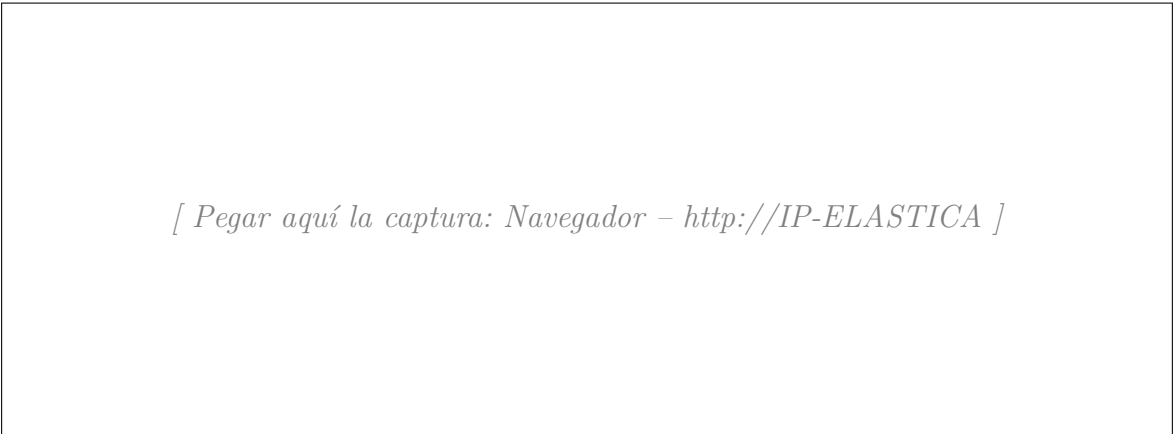


Figura 17: Security Group (puertos 22, 80).



[Pegar aquí la captura: Terminal EC2 – docker compose ps]

Figura 18: Contenedores en ejecución (`docker compose ps`).



[Pegar aquí la captura: Navegador – http://IP-ELASTICA]

Figura 19: Aplicación funcionando por la IP elástica.

18. Repositorios públicos de GitHub

- Backend: <https://github.com/UTEC-AII/finzen-app>
- Frontend: <https://github.com/UTEC-AII/finzen-webui>

19. Conclusiones y trabajo futuro

FinZen satisface el alcance del MVP propuesto: registro seguro de usuarios, captura de ingresos y gastos, listados y panel de control, y un asistente basado en RAG que responde sobre los datos del propio usuario. Se aplicaron prácticas de contenedores (imágenes livianas, usuario no privilegiado, *healthchecks* y volúmenes) y de seguridad (JWT, bcrypt y CORS restringido).

Trabajo futuro: escalar horizontalmente mediante **Application Load Balancer** y **Auto Scaling**, migrar a **PostgreSQL en RDS**, incorporar **reranking** y **sqlite-vec** en el pipeline de RAG, y automatizar el despliegue mediante CI/CD.

Referencias

- Documentación de Docker y Docker Compose.
- Documentación de FastAPI y Pydantic.
- Documentación de AWS EC2 y calculadora de costos (<https://calculator.aws>).
- Documentación de OpenAI (modelos de embeddings y de chat).
- Documentación de Next.js (App Router).

Anexos

- Anexo A: prompt del prototipo (`frontend-prompt/prompt.md`).
- Anexo B: colección Postman y entornos (`postman/`).
- Anexo C: catálogo completo de APIs (`CATALOGO_DE_APIS.md`).